

Evaluación Continua 3

Se cuenta con un *scanner*, *parser* y *visitor* que trabajan sobre la siguiente gramática:

$$\begin{aligned} \textit{Program} &::= \textit{StmtList} \\ \textit{StmtList} &::= \textit{Stmt} \{ ; \textit{Stmt} \}^* \\ \textit{Stmt} &::= \textit{Id} = \textit{CExp} \mid \text{print} (\textit{CExp}) \\ \textit{CExp} &::= \textit{Expr} \{ (+ \mid -) \textit{Expr} \}^* \\ \textit{Expr} &::= \textit{Term} \{ (* \mid /) \textit{Term} \}^* \\ \textit{Term} &::= \textit{Factor} [* \textit{Factor}] \\ \textit{Factor} &::= \textit{Number} \mid (\textit{CExp}) \mid \text{sqrt}(\textit{CExp}) \mid \textit{Id} \end{aligned}$$

Esta gramática define un mini-lenguaje imperativo que permite construir programas compuestos por una lista de sentencias separadas por punto y coma. Dichas sentencias pueden ser asignaciones de valores a variables mediante expresiones aritméticas o instrucciones de salida con **print**. Ejemplos de programas válidos:

1. `x = 5; y = 10; print(x + y)`
2. `a = 2 ** 3; b = sqrt(16); print(a * b)`
3. `m = (3 + 7) * 2; n = m - 5; print(n)`
4. `u = 100 / 4; v = u + 6; w = v ** 2; print(w)`
5. `x = sqrt(25); y = (x + 3) * 2; print(y - 1)`
6. `a = 7; b = 8; c = (a + b) / 3; print(c)`
7. `r = 2; s = r ** 4; t = sqrt(s); print(t)`
8. `p = 9; q = p - 4; r = q * (2 + 3); print(r)`

La gramática debe ser extendida de modo que el *scanner*, el AST, el *parser* y el *visitor* soporten las siguientes construcciones adicionales:

$$\begin{aligned} \textit{Program} &::= \textit{StmtList} \\ \textit{StmtList} &::= \textit{Stmt} \{ ; \textit{Stmt} \}^* \\ \textit{Stmt} &::= \textit{Id} = \textit{CExp} \mid \text{print} (\textit{PrintArg} \{ , \textit{PrintArg} \}^*) \\ &::= \textit{Id} \{ , \textit{Id} \}^* = (\textit{Cexp} \{ , \textit{Cexp} \}^*) \\ \textit{CExp} &::= \textit{Expr} \{ (+ \mid -) \textit{Expr} \}^* \\ \textit{Expr} &::= \textit{Term} \{ (* \mid /) \textit{Term} \}^* \\ \textit{Term} &::= \textit{Factor} [* \textit{Factor}] \\ \textit{Factor} &::= \textit{Number} \mid (\textit{CExp}) \mid \text{sqrt}(\textit{CExp}) \mid \textit{Id} \\ &::= \text{min} (\textit{Cexp} \{ , \textit{Cexp} \}^*) \\ \textit{PrintArg} &::= \textit{CExp} \mid \textit{String} \\ \textit{String} &::= \textit{id} \end{aligned}$$

Ejemplos de cadenas válidas con las nuevas reglas:

1. `x = 5; print(x)`
2. `x = 5; y = 8; print('x=', x, ' y=', y)`
3. `x = sqrt(25); y = 3 + 2 * 4; print('x=', x, ' y=', y)`
4. `m = min(7,3); print(m)`
5. `x = 10; y = 4; m = min(x,y); print(m)`
6. `x = 8; y = 4; u, v = (x+y, x-y); print(u+v)`
7. `p = min(3,7,2,9,4); q = sqrt((p+6)*(p+6)); print(p+q)`
8. `x, y, z = (2,3,4); a = x ** y; b = sqrt(z*z + a); print('suma', a+b)`
9. `m = 7; n = 2; r1, r2 = (m*n, m/n); print('resultado', min(r1,r2,m+n))`
10. `a = 4; b = 9; c = 16; x, y = (sqrt(a)+sqrt(b), min(a,b,c)); print('x=', x, ' y=', y)`

Pregunta

Explique los cambios que deben incorporarse en el scanner, el parser, el AST y el visitor para soportar la asignación múltiple en el lenguaje. Describa al menos diez pasos claramente enumerados que indiquen las modificaciones necesarias en cada uno de estos componentes del compilador.